

USB2UARTSPIIICDLL 库函数说明

V3.1.2

目录

一 说明	4
二 宏定义	4
三 打开关闭 USB，获取转接板信息相关函数	6
1. 打开 USB 函数：OpenUsb	6
2. 关闭 USB 函数：CloseUsb	6
3. 读取转接板第一次上电标志函数：IsFirstPowerUp	7
4. 获取转接板信息函数：GetBoardInformation	7
5. 举例	8
四 UART 相关函数	8
1. 设置 UART 参数函数：ConfigUARTParam	8
2. UART 发送数据函数：UARTSendData	9
3. UART 接收数据函数：UARTRcvData	10
4. 举例	10
五 SPI 主模式相关函数	11
1. 设置 SPI 主模式参数函数：ConfigSPIParam	11
2. 设置 SPI CS0 管脚电平函数：SPISetCS0	12
3. 设置 SPI CS1 管脚电平函数：SPISetCS0	12
4. SPI 主模式发送接收数据函数：SPISendAndRcvData	13
5. SPI 主模式发送数据函数：SPISendData	14
5. SPI 主模式读取数据函数：SPIRcvData	15
6. 举例	15
六 SPI 从模式相关函数	16
1. 设置 SPI 从模式参数函数：ConfigSPIParamSlave	16
2. SPI 从模式预装数据函数：SPISlavePreloadData	17
3. SPI 从模式读取数据函数：SPISlaveRcvData	17
4 举例	18
七 I2C 主模式相关的函数	18
1. 设置 I2C 主模式参数函数：ConfigIICParam	18
2. I2C 主模式发送接收数据函数：IICSendAndRcvData	19

3. I2C 主模式发送数据函数: IICSendData	20
4. I2C 主模式接收数据函数: IICRcvData	21
5. I2C 主模式检测从机地址: IICCheckSlaveAddr	22
6. I2C 主模式寄存器发送函数: IICRegisterSend	22
7. I2C 主模式寄存器读取函数: IICRegisterRead	23
8. I2C 主模式直接发送函数: IICDirectSend	24
9. I2C 主模式直接读取函数: IICDirectRead	25
10. 举例	26
八 I2C 从模式相关函数	27
1. 设置 I2C 从模式参数函数: ConfigIICParamSlave	27
2. I2C 从模式预装数据函数: IICSlavePreloadData	27
3. I2C 从模式读取数据函数: IICSlaveRcvData	28
4. 举例	29
九 ADC 相关函数	29
1. ADC 读取函数: GetADCVal	29
2. 举例	30
十 IO 口相关函数	30
1. IO 口读写函数: IOSetAndRead	30
2. 举例	31
十一 PWM 相关函数	32
1. PWM 输出函数: PWMOut	32
2. 关闭 PWM 输出函数: PWMClose	32
3. 举例	33
十二 内建列表相关函数（仅快速版支持）	33
1. 初始化内建列表函数: InternalListInitAllLine	33
2. 设置内建列表某一行函数: InternalListSetLine	34
3. 失能内建列表某一行函数: InternalListDisableLine	36
4. 测试执行一次内建列表函数: InternalListTsetOnce	36
5. I00 跳变沿触发执行内建列表函数: InternalListI00Start	37
6. 定时器沿触发执行内建列表函数: InternalListI00Start	37
7. 内建列表接收数据函数: InternalListReceive	38
8. 停止执行内建列表函数: InternalListStop	38

9. 举例39

一 说明

USB2UARTSPIIICDLL 动态链接库支持动态调用和静态调用，支持 32 位和 64 位。如果在多线程调用本动态链接库中的函数，需要设置线程互斥，确保我们的库函数在运行的过程中不被我们库中别的函数打断。

USB2UARTSPIIICDLL 动态链接库支持 C++，C#，java，Python，MATLAB，labview 等语言调用。

函数返回值说明：

所有函数的返回值都是 int 型，返回值大于等于 0，表示函数执行正确。返回为负数，表示函数执行错误。返回-1 表示还没有打开 USB，返回其它负数，一般会引起丢数据，或者需要重启转接板才能恢复故障。快速版一般会自动重启。基础版和多电压版需要手动重新插拔一下重启。

函数中 UsbIndex 参数的说明：

如果电脑上只连接了一个转接板，这个参数使用默认值 0。如果同一台电脑上连接了两个转接板，这个参数就是 0 和 1。以此类推。UsbIndex 得取值范围是 0 到 99。

二 宏定义

```
//UART 奇偶校验位
#define UART_Parity_No           0
#define UART_Parity_Odd         1
#define UART_Parity_Even        2

//UART 停止位
#define UART_StopBits_1          0
#define UART_StopBits_1_5        1
#define UART_StopBits_2          2

//SPI 速率
#define SPI_Rate_281K             0
#define SPI_Rate_562K             1
#define SPI_Rate_1_125M           2
#define SPI_Rate_2_25M            3
#define SPI_Rate_4_5M             4
#define SPI_Rate_9M               5
#define SPI_Rate_18M              6
#define SPI_Rate_36M              7

//SPI 帧格式
#define SPI_MSB                   0
#define SPI_LSB                   1
```

```

//SPI 时钟模式
#define SPI_SubMode_0          0
#define SPI_SubMode_1          1
#define SPI_SubMode_2          2
#define SPI_SubMode_3          3

//IIC 速率
#define IIC_Rate_1K             0
#define IIC_Rate_5K             1
#define IIC_Rate_10K            2
#define IIC_Rate_20K            3
#define IIC_Rate_50K            4
#define IIC_Rate_80K            5
#define IIC_Rate_100K           6
#define IIC_Rate_200K           7
#define IIC_Rate_400K           8
#define IIC_Rate_600K           9
#define IIC_Rate_800K          10
#define IIC_Rate_1M            11

//IIC 寻址模式
#define IIC_ADDRMOD_7BIT        0
#define IIC_ADDRMOD_10BIT       1

//内建列表功能 1
#define LINE_FUN1_SPI_CS0_0_1    0
#define LINE_FUN1_SPI_CS0_1_0    1
#define LINE_FUN1_SPI_CS0_1_1    2
#define LINE_FUN1_SPI_CS0_0_0    3
#define LINE_FUN1_SPI_CS1_0_1    4
#define LINE_FUN1_SPI_CS1_1_0    5
#define LINE_FUN1_SPI_CS1_1_1    6
#define LINE_FUN1_SPI_CS1_0_0    7
#define LINE_FUN1_SPICS_0         8
#define LINE_FUN1_SPICS_1         9
#define LINE_FUN1_IIC_AddStart    10
#define LINE_FUN1_IIC_NoStart     11
#define LINE_FUN1_IICd_send       12
#define LINE_FUN1_IICd_read       13
#define LINE_FUN1_IICr_send       14
#define LINE_FUN1_IICr_read       15
#define LINE_FUN1_UART_           16
#define LINE_FUN1_IO_0            17
#define LINE_FUN1_IO_1            18
#define LINE_FUN1_PWM_start       19
#define LINE_FUN1_PWM_stop        20
#define LINE_FUN1_ADC_0           21
#define LINE_FUN1_ADC_1           22

//内建列表功能 2

```

```
#define LINE_FUN2_null 0
#define LINE_FUN2_SPI_full 1
#define LINE_FUN2_SPI_half 2
#define LINE_FUN2_SPICS_h 3
#define LINE_FUN2_SPICS_l 4
#define LINE_FUN2_IIC_AddStop 5
#define LINE_FUN2_IIC_NoStop 6
#define LINE_FUN2_IICd_7bit 7
#define LINE_FUN2_IICd_10bit 8
#define LINE_FUN2_IICr_7bit 9
#define LINE_FUN2_IICr_10bit 10
#define LINE_FUN2_IO_w 11
#define LINE_FUN2_IO_r 12
```

三 打开关闭 USB，获取转接板信息相关函数

1. 打开 USB 函数：OpenUsb

函数原型：int OpenUsb(unsigned int UsbIndex)

函数说明：通过索引值打开转接板的USB。调用此函数来初始化转接板的USB。通过不同的索引值来区分同一台电脑上的多个转接板。

参数：

UsbIndex	USB 索引值（0-99）
----------	---------------

返回值：

0	成功打开 USB
-1	同一索引值的 USB 被重复打开，USB 未插入电脑，USB 被其它程序占用

2. 关闭 USB 函数：CloseUsb

函数原型：int CloseUsb(unsigned int UsbIndex)

函数说明：通过索引值关闭转接板的USB。调用此函数来关闭转接板的USB，释放对该索引值USB的占用。

参数：

UsbIndex	USB 索引值（0-99）
----------	---------------

返回值:

0	成功关闭 USB
---	----------

3. 读取转接板第一次上电标志函数: IsFirstPowerUp

函数原型: int IsFirstPowerUp (unsigned int UsbIndex)

函数说明: 转接板刚上电, 未进行任何设置参数的操作, 该函数返回1。一旦设置了UART, SPI, IIC, IO, PWM, ADC参数, 该函数返回0。该函数一般用来判断转接板是不是刚上电未配置状态, 是否被重新插拔过。

参数:

UsbIndex	USB 索引值 (0-99)
----------	----------------

返回值:

0	USB 转接板已经被配置过参数
1	USB 转接板上电, 未被配置参数
-1	USB 未打开
小于-1	协议错误

4. 获取转接板信息函数: GetBoardInformation

函数原型: int GetBoardInformation (unsigned char *UIDBuf, unsigned int *isHsUSB, unsigned int *FwVersion, unsigned int UsbIndex)

函数说明: 获取转接板的UID, USB速度, 固件版本信息。UID是一个96bit的唯一ID, 共12个字节, 每个转接板的UID都不相同。可以通过USB速度来区分转接板是类型, 0表示全速USB2.0, 1表示高速USB2.0。基础版和多电压版是的USB是全速USB2.0, 快速版的USB是高速USB2.0。固件版本是一个正整数, 这个正整数除以100, 表示当前固件版本号, 例如310, 表示固件版本号是V3.10。

参数:

*UIDBuf	返回 UID
*isHsUSB	返回 USB 速度类型
*FwVersion	返回固件版本
UsbIndex	USB 索引值 (0-99)

返回值:

0	读取转接板信息成功
-1	USB 未打开
小于-1	协议错误

5. 举例

```
unsigned char UID[12];           //UID
unsigned int isHsUSB;             //USB 速度类型
unsigned int firmwareVersion;    //固件版本号
//打开索引值为 0 的转接板 USB
OpenUsb(0);
//打开索引值为 1 的转接板 USB
OpenUsb(1);

//读取转接板第一次上电标志（这个函数根据实际需要调用，不需要就不调用）
if(IsFirstPowerUp(0) == 1)
{
    //索引值为 0 的转接板刚上电
}
else if(IsFirstPowerUp(0) == 0)
{
    //索引值为 0 的转接板已经配置过参数
}

//获取转接板 0 的 UID，USB 类型，固件版本信息
GetBoardInformation(UID,&isHsUSB,&firmwareVersion,0);

//获取转接板 1 的 UID，USB 类型，固件版本信息
GetBoardInformation(UID,&isHsUSB,&firmwareVersion,1);

/*****/
//这里调用 UART，SPI，IIC 等相关操作函数
/*****/

//关闭索引值为 0 的转接板 USB
CloseUsb(0);
//关闭索引值为 1 的转接板 USB
CloseUsb(1);
```

四 UART 相关函数

1. 设置 UART 参数函数：ConfigUARTParam

函数原型：int ConfigUARTParam (unsigned int baudRate,unsigned int parity,

unsigned int stopBits,unsigned int UsbIndex)

函数说明：设置转接板UART的参数，包括波特率，校验位，停止位。基础版和多电压版最高波特率115200，多电压版最高波特率4.5M。

参数：

baudRate	波特率：基础版和多电压版的取值范围是2400到115200。 多电压版的取值范围是2400到4500000。
parity	校验位：UART_Parity_No 无校验 UART_Parity_Even 偶校验 UART_Parity_Odd 奇校验
stopBits	停止位：UART_StopBits_1 1位 UART_StopBits_1_5 1.5位 UART_StopBits_2 2位
UsbIndex	USB索引值（0-99）

返回值：

0	UART参数成功
-1	USB未打开
小于-1	协议错误

2. UART 发送数据函数：UARTSendData

函数原型：int UARTSendData(unsigned char *sendBuf, unsigned int len,
unsigned int UsbIndex)

函数说明：UART发送数据函数。基础版和多电压版每次最多发送1024字节。快速版每次最多发送4096字节。如果基础版和多电压版设置发送数据长度大于1024，该函数只发送前1024字节。如果快速版版设置发送数据长度大于4096，该函数只发送前4096字节。多出来的数据会被忽略。

参数：

sendBuf	发送数据的缓存
len	发送数据的缓存的长度。基础版和多电压版最多1024字节。快速版最多4096字节。
UsbIndex	USB索引值（0-99）

返回值：

0	数据发送成功
---	--------

-1	USB 未打开
小于-1	协议错误

3. UART 接收数据函数：UARTRcvData

函数原型： int UARTRcvData(unsigned char *rcvBuf,unsigned int len,unsigned int UsbIndex)

函数说明： UART接收数据函数。实际转接板已经接收到数据，并存放于转接板的缓存中。这个函数是读取缓存中的数据，最多读取len个字节。如果缓存中数据不够len个字节，则返回实际读取到数据的长度。如果缓存中大于len个字节，则返回len个字节，剩下的数据需要再一次调用UARTRcvData函数才能读出。
基础版和多电压版每次最多读取1024字节。快速版每次最多读取65535字节。

参数：

rcvBuf	接收数据的缓存
len	接收数据的缓存的长度。基础版和多电压版最多1024字节。快速版最多65535字节。
UsbIndex	USB 索引值（0-99）

返回值：

大于等于 0	数据读取成功
-1	USB 未打开
小于-1	协议错误

4. 举例

```
//发送的缓存
unsigned char sendBuf[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//接收的缓存
unsigned char receiveBuf[10];

//打开索引值为 0 的转接板 USB
OpenUsb(0);

//设置 UART 参数：波特率 115200，无校验，1 位停止位
ConfigUARTParam(115200,UART_Parity_No,UART_StopBits_1,0);

//UART 发送 10 个字节
UARTSendData(sendBuf, 10, 0);
```

```
//关闭索引值为 0 的转接板 USB  
CloseUsb(0);
```

2. 设置 SPI CS0 管脚电平函数：SPISetCS0

函数原型： int SPISetCS0(unsigned char cs,unsigned int UsbIndex)

函数说明： 设置转接板SPI CS0管脚电平。

参数：

cs	CS0管脚电平： 0 1	低电平 高电平
UsbIndex	USB 索引值（0-99）	

返回值：

0	CS0 管脚电平设置成功
-1	USB 未打开
小于-1	协议错误

3. 设置 SPI CS1 管脚电平函数：SPISetCS0

函数原型： int SPISetCS1(unsigned char cs,unsigned int UsbIndex)

函数说明： 设置转接板SPI CS1管脚电平。

参数：

cs	CS1管脚电平： 0 1	低电平 高电平
UsbIndex	USB 索引值（0-99）	

返回值：

0	CS1 管脚电平设置成功
-1	USB 未打开
小于-1	协议错误

4. SPI 主模式发送接收数据函数：SPISendAndRcvData

函数原型：int SPISendAndRcvData (unsigned char CSselect,unsigned char startCS,
unsigned char endCS, unsigned int CSdelay,
unsigned char duplex,unsigned char dummy,
unsigned char *sendBuf,unsigned char *rcvBuf,
unsigned int slen,unsigned int rlen,
unsigned int UsbIndex)

函数说明：SPI主模式发送和接收数据。基础版和多电压版发送和接收数据之和不能大于1024字节。快速版发送和接收数据之和不能大于8192字节。

参数：

CSselect	CS管脚选择： 0 使用CS0脚 1 使用 CS1 脚
startCS	本次传输数据前CS电平： 0 传输数据前拉低CS脚 1 传输数据前拉高CS脚
endCS	本次传输数据完成后CS电平： 0 传输数据完成后拉低CS脚 1 传输数据完成后拉高CS脚
CSdelay	CS延时： 数据开始传输前，CS 脚电平跳变沿到SCK第一个 时钟跳变沿的延时。单 位： 微秒。最大延时 700000微秒。
duplex	通讯方式： 0 半双工 1 全双工
dummy	接收时虚拟字节：SPI在接收数据时，MOSI发 出的数据。
*sendBuf	发送数据的缓存
*rcvBuf	接收数据的缓存
slen	发送数据的长度：基础版和多电压版每次最 多发送1024字节，且必须 发送数据和读取数据的总 和小于等于1024字节。快 速版每次最多发送8192字 节，且必须发送数据和读 取数据的总和小于等于 8192字节。
rlen	读取数据的长度：基础版和多电压版每次最 多读取1024字节，且必须

	发送数据和读取数据的总和小于等于1024字节。快速版每次最多读取8192字节，且必须发送数据和读取数据的总和小于等于8192字节。
UsbIndex	USB 索引值（0-99）

返回值：

大于等于 0	发送和读取数据成功。返回实际读取的字节数。
-1	USB 未打开
小于-1	协议错误

5. SPI 主模式发送数据函数：SPISendData

函数原型： int SPISendData(unsigned int startCS, unsigned int endCS,
 unsigned char *sendBuf, unsigned int len,
 unsigned int UsbIndex)

函数说明： SPI主模式发送数据。基础版和多电压版每次最多发送1024字节。快速版每次最多发送8192字节。使用CS0，CS0延时200微秒。

参数：

startCS	本次传输数据前CS电平： 0 传输数据前拉低CS脚 1 传输数据前拉高CS脚
endCS	本次传输数据完成后CS电平： 0 传输数据完成后拉低CS脚 1 传输数据完成后拉高CS脚
*sendBuf	发送数据的缓存
slen	发送数据的长度：基础版和多电压版每次最多发送1024字节。快速版每次最多发送8192字节。
UsbIndex	USB 索引值（0-99）

返回值：

0	发送数据成功
-1	USB 未打开
小于-1	协议错误

5. SPI 主模式读取数据函数：SPIRcvData

函数原型：int SPIRcvData(unsigned int startCS, unsigned int endCS,
 unsigned char *rcvBuf, unsigned int len,
 unsigned int UsbIndex)

函数说明：SPI主模式读取数据。基础版和多电压版每次最多读取1024字节。快速版每次最多读取8192字节。使用CS0，CS0延时200微秒。接收时虚拟字节0xFF。

参数：

startCS	本次传输数据前CS电平： 0 传输数据前拉低CS脚 1 传输数据前拉高CS脚
endCS	本次传输数据完成后CS电平： 0 传输数据完成后拉低CS脚 1 传输数据完成后拉高CS脚
*rcvBuf	读取数据的缓存
len	读取数据的长度：基础版和多电压版每次最 读取送1024字节。快速版 每次最多读取8192字节。
UsbIndex	USB 索引值（0-99）

返回值：

大于等于 0	读取数据成功，返回实际读取的字节数。
-1	USB 未打开
小于-1	协议错误

6. 举例

```
//发送的缓存
unsigned char sendBuf[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//接收的缓存
unsigned char receiveBuf[10];

//打开索引值为 0 的转接板 USB
OpenUsb(0);

//设置 SPI 主模式参数：18M，高位在前，模式 0
ConfigSPIParam(SPI_Rate_18M, SPI_MSB, SPI_SubMode_0, 0);
```

```
//设置 CS0 高电平
SPISetCS0(1);

//SPI 发送和接收 10 个字节，使用 CS0，传输数据前拉低 CS 脚，传输数据完成后拉高 CS 脚，
CS 脚延时 50 微秒，全双工模式，接收时虚拟字节 0xFF
SPISendAndRcvData (0,0,1, 50,1,0xFF, sendBuf, receiveBuf, 10, 10, 0);

//SPI 发送 10 个字节，传输数据前拉低 CS0 脚，传输数据完成后拉高 CS0 脚
SPISendData(0, 1, sendBuf, 10, 0);

//SPI 读取 10 个字节，传输数据前拉低 CS0 脚，传输数据完成后拉高 CS0 脚
SPIRcvData(0, 1, receiveBuf, 10, 0);

//关闭索引值为 0 的转接板 USB
CloseUsb(0);
```

六 SPI 从模式相关函数

1. 设置 SPI 从模式参数函数：ConfigSPIParamSlave

函数原型：int ConfigSPIParamSlave(unsigned int fistBit,unsigned int subMode,
unsigned int UsbIndex)

函数说明：设置转接板SPI从模式的参数，包括帧格式，时钟模式。基础版和多电压版支持主机的时钟频率最高到9M，快速版支持主机的时钟频率最高到36M。

参数：

fistBit	帧格式： SPI_MSB SPI_LSB	高位在前 低位在前
subMode	时钟模式： SPI_SubMode_0 SPI_SubMode_1 SPI_SubMode_2 SPI_SubMode_3	CPOL=0， CPHA=0 CPOL=0， CPHA=1 CPOL=1， CPHA=0 CPOL=1， CPHA=1
UsbIndex	USB 索引值（0-99）	

返回值：

0	SPI 主模式参数成功
-1	USB 未打开
小于-1	协议错误

2. SPI 从模式预装数据函数：SPISlavePreloadData

函数原型：int SPISlavePreloadData(unsigned char *pBuf, unsigned int len,
unsigned int UsbIndex)

函数说明：SPI从模式预装数据。通过这个函数将数组预装到转接板，然后SPI主机就能把预装的数据读出。基础版和多电压版每次最多预装1024字节。快速版每次最多预装8192字节。

参数：

*pBuf	预装数据的缓存
len	预装数据的长度：基础版和多电压版每次最多预装1024字节。快速版每次最多预装8192字节。
UsbIndex	USB 索引值（0-99）

返回值：

0	预装数据成功
-1	USB 未打开
小于-1	协议错误

3. SPI 从模式读取数据函数：SPISlaveRcvData

函数原型：int SPISlaveRcvData(unsigned char *rcvBuf, unsigned int len,
unsigned int UsbIndex)

函数说明：SPI从模式读取数据。实际转接板已经接收到数据，并存放到转接板的缓存中。这个函数是读取缓存中的数据，最多读取len个字节。如果缓存中数据不够len个字节，则返回实际读取到数据的长度。如果缓存中大于len个字节，则返回len个字节，剩下的数据需要再一次调用SPISlaveRcvData函数才能读出。基础版和多电压版每次最多读取1024字节。快速版每次最多读取65535字节。

参数：

*rcvBuf	读取数据的缓存
len	预装数据的长度：基础版和多电压版每次最多读取1024字节。快速版

	每次最多读取65535字节。
UsbIndex	USB 索引值（0-99）

返回值：

大于等于 0	读取数据成功
-1	USB 未打开
小于-1	协议错误

4 举例

```
//发送的缓存
unsigned char sendBuf[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//接收的缓存
unsigned char receiveBuf[10];

//打开索引值为 0 的转接板 USB
OpenUsb(0);

//设置 SPI 从模式参数：高位在前，模式 0
ConfigSPIParamSlave(SPI_MSB, SPI_SubMode_0, 0);

//SPI 从机预装 10 个字节
SPISlavePreloadData (sendBuf, 10, 0);

//SPI 从机读取 10 个字节
SPISlaveRcvData(receiveBuf, 10, 0);

//关闭索引值为 0 的转接板 USB
CloseUsb(0);
```

七 I2C 主模式相关的函数

1. 设置 I2C 主模式参数函数：ConfigIICParam

函数原型： int ConfigIICParam(unsigned int rate,unsigned int clkSLevel,
unsigned int UsbIndex)

函数说明： 设置转接板I2C主模式的参数，包括时钟频率，时钟延展。基础版和多电压版支持的时钟频率最高到400K，快速版支持的时钟频率最高到1M。

参数：

rate	时钟频率： IIC_Rate_1K 1K IIC_Rate_5K 5K IIC_Rate_10K 10K IIC_Rate_20K 20K IIC_Rate_50K 50K IIC_Rate_80K 80K IIC_Rate_100K 100K IIC_Rate_200K 200K IIC_Rate_400K 400K IIC_Rate_600K 600K（仅快速版） IIC_Rate_800K 800K（仅快速版） IIC_Rate_1M 1M （仅快速版）
clkSLevel	时钟延展： IIC时钟延展等待的时钟周期。可以设置等待0到4294967295 (0xFFFFFFFF)个时钟周期，设置成4294967295会无限等待
UsbIndex	USB 索引值（0-99）

返回值：

0	I2C 主模式参数成功
-1	USB 未打开
小于-1	协议错误

2. I2C 主模式发送接收数据函数：IICSendAndRcvData

函数原型： int IICSendAndRcvData(unsigned char startBit,unsigned char stopBit,
 unsigned char *sendBuf,unsigned char *rcvBuf,
 unsigned int slen,unsigned int rlen,
 unsigned int UsbIndex)

函数说明： I2C主模式发送和接收数据。基础版和多电压版发送和接收数据之和不能大于1024字节。快速版发送和接收数据之和不能大于2048字节。

参数：

startBit	START信号： 0 执行本次传输前不加Strat
----------	---

	1 执行本次传输前加 Strat
stopBit	STOP信号: 0 执行本次传输后不加Stop 1 执行本次传输后加Stop
*sendBuf	发送数据的缓存
*rcvBuf	接收数据的缓存
slen	发送数据的长度: 基础版和多电压版每次最多发送1024字节, 且必须发送数据和读取数据的总和小于等于1024字节。快速版每次最多发送2048字节, 且必须发送数据和读取数据的总和小于等于2048字节。
rlen	读取数据的长度: 基础版和多电压版每次最多读取1024字节, 且必须发送数据和读取数据的总和小于等于1024字节。快速版每次最多读取2048字节, 且必须发送数据和读取数据的总和小于等于2048字节。
UsbIndex	USB 索引值 (0-99)

返回值:

大于等于 0	发送和读取数据成功。返回实际读取的字节数。
-1	USB 未打开
小于-1	协议错误

3. I2C 主模式发送数据函数: IICSendData

函数原型: int IICSendData(unsigned char startBit,unsigned char stopBit,
unsigned char *sendBuf,unsigned int slen,
unsigned int UsbIndex)

函数说明: I2C主模式发送数据。基础版和多电压版发每次最多发送1024字节。快速版每次最多发送2048字节。

参数:

startBit	START信号: 0 执行本次传输前不加Strat
----------	------------------------------

	1 执行本次传输前加 Strat
stopBit	STOP信号: 0 执行本次传输后不加Stop 1 执行本次传输后加Stop
*sendBuf	发送数据的缓存
slen	发送数据的长度: 基础版和多电压版每次最多发送1024字节。快速版每次最多发送2048字节。
UsbIndex	USB 索引值 (0-99)

返回值:

大于等于 0	发送和读取数据成功。返回实际读取的字节数。
-1	USB 未打开
小于-1	协议错误

4. I2C 主模式接收数据函数: IICRcvData

函数原型: int IICRcvData(unsigned char stopBit,unsigned char *rcvBuf,
unsigned int rlen,unsigned int UsbIndex)

函数说明: I2C主模式读取数据。基础版和多电压版每次最多读取1024字节。快速版每次最多读取2048字节。

参数:

stopBit	STOP信号: 0 执行本次传输后不加Stop 1 执行本次传输后加Stop
*rcvBuf	接收数据的缓存
rlen	读取数据的长度: 基础版和多电压版每次最多读取1024字节。快速版每次最多读取2048字节。
UsbIndex	USB 索引值 (0-99)

返回值:

大于等于 0	读取数据成功。返回实际读取的字节数。
-1	USB 未打开
小于-1	协议错误

5. I2C 主模式检测从机地址：IICCheckSlaveAddr

函数原型：int IICCheckSlaveAddr(unsigned char addrMod,unsigned int addr,
unsigned int UsbIndex)

函数说明：I2C主模式检测从机地址函数。本函数用来检测从机地址是否正确，如果从机地址正确返回1，如果从机地址错误返回0。注意这里只有返回1的时候是正确的，返回负数的时候是不正确的。

参数：

addrMod	地址模式： 0 7Bit地址 1 10Bit 地址
addr	从机地址：7Bit地址模式下地址范围是0x00到0x7F。 10Bit地址模式下地址范围是0x000到0x3FF。
UsbIndex	USB 索引值（0-99）

返回值：

0	没有检测到从机地址
1	检测到从机地址
-1	USB 未打开
小于-1	协议错误

6. I2C 主模式寄存器发送函数：IICRegisterSend

函数原型：int IICRegisterSend(unsigned char addrMod,unsigned int addr,
unsigned char *regBuf,unsigned char *sendBuf,
unsigned char reglen,unsigned int slen,
unsigned int UsbIndex)

函数说明：I2C主模式寄存器发送函数。基础版和多电压版发每次最多发送1024字节。快速版每次最多发送2048字节。基础版，多电压版和快速版寄存器地址最多58字节。并且基础版和多电压版寄存器地址长度加数据的长度不能大于1024字节。快速版寄存器地址长度加数据的长度不能大于2048字节。

时序：

Start	从机地址+读写位0	发送寄存器地址	发送的数据	Stop
-------	-----------	---------	-------	------

参数:

addrMod	地址模式： 0 7Bit地址 1 10Bit 地址
addr	从机地址：7Bit地址模式下地址范围是0x00到0x7F。 10Bit地址模式下地址范围是0x000到0x3FF。
*regBuf	寄存器地址缓存
*sendBuf	发送数据的缓存
reglen	寄存器地址长度，寄存器地址最多58字节
slen	发送数据的长度：基础版和多电压版每次最多发送1024字节，且寄存器地址长度加数据的长度不能大于1024字节。快速版每次最多发送2048字节，且寄存器地址长度加数据的长度不能大于2048字节。
UsbIndex	USB 索引值（0-99）

返回值:

0	发送数据成功。
-1	USB 未打开
小于-1	协议错误

7. I2C 主模式寄存器读取函数: IICRegisterRead

函数原型: int IICRegisterRead(unsigned char addrMod,unsigned int addr,
unsigned char *regBuf,unsigned char *rcvBuf,
unsigned char reglen,unsigned int rlen,
unsigned int UsbIndex)

函数说明：I2C主模式寄存器读取函数。基础版和多电压版发每次最多读取1024字节。快速版每次最多读取2048字节。基础版，多电压版和快速版寄存器地址最多58字节。

时序:

Start	从机地址+读写位0	寄存器地址	ReStart	从机地址+读写位1	读取的数据	Stop
-------	-----------	-------	---------	-----------	-------	------

参数:

addrMod	地址模式: 0 7Bit地址 1 10Bit 地址
addr	从机地址: 7Bit地址模式下地址范围是0x00到0x7F。 10Bit地址模式下地址范围是0x000到0x3FF。
*regBuf	寄存器地址缓存
*rcvBuf	读取数据的缓存
reglen	寄存器地址长度, 寄存器地址最多58字节
rlen	读取数据的长度: 基础版和多电压版每次最多发送1024字节。快速版每次最多发送2048字节。
UsbIndex	USB 索引值 (0-99)

返回值:

大于等于 0	读取数据成功。返回实际读取的字节数。
-1	USB 未打开
小于-1	协议错误

8. I2C 主模式直接发送函数: IICDirectSend

函数原型: int IICDirectSend(unsigned char addrMod,unsigned int addr,
 unsigned char *sendBuf,unsigned int slen,
 unsigned int UsbIndex)

函数说明: I2C主模式直接发送函数。基础版和多电压版发每次最多发送1024字节。快速版每次最多发送2048字节。

时序:

Start	从机地址+读写位0	发送的数据	Stop
-------	-----------	-------	------

参数:

addrMod	地址模式: 0 7Bit地址 1 10Bit 地址
addr	从机地址: 7Bit地址模式下地址范围是0x00到0x7F。 10Bit地址模式下地址范围是

	0x000到0x3FF。
*sendBuf	发送数据的缓存
slen	发送数据的长度：基础版和多电压版每次最多发送1024字节。快速版每次最多发送2048字节。
UsbIndex	USB 索引值（0-99）

返回值：

大于等于 0	发送数据成功。
-1	USB 未打开
小于-1	协议错误

9. I2C 主模式直接读取函数：IICDirectRead

函数原型： int IICDirectRead(unsigned char addrMod,unsigned int addr,
unsigned char *rcvBuf,unsigned int rlen,
unsigned int UsbIndex)

函数说明： I2C主模式直接读取函数。基础版和多电压版发每次最多读取1024字节。快速版每次最多读取2048字节。

时序：

Start	从机地址+读写位1	读取的数据	Stop
-------	-----------	-------	------

参数：

addrMod	地址模式： 0 7Bit地址 1 10Bit 地址
addr	从机地址：7Bit地址模式下地址范围是0x00到0x7F。 10Bit地址模式下地址范围是0x000到0x3FF。
*rcvBuf	读取数据的缓存
rlen	读取数据的长度：基础版和多电压版每次最多发送1024字节。快速版每次最多发送2048字节。
UsbIndex	USB 索引值（0-99）

返回值：

大于等于 0	读取数据成功。返回实际读取的字节数。
--------	--------------------

-1	USB 未打开
小于-1	协议错误

10. 举例

```

//发送的缓存
unsigned char sendBuf[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//寄存器的缓存
unsigned char regBuf[2] = {0x00, 0x01};
//接收的缓存
unsigned char receiveBuf[10];

//打开索引值为 0 的转接板 USB
OpenUsb(0);

//设置 I2C 主模式参数：时钟频率 400K，时钟延展 10000 个时钟周期
ConfigIICParam(IIC_Rate_400K, 10000, 0);

//发送前加 START, 发送完成后加 STOP，发送 10 个字节，接收 10 个字节
IICSendAndRcvData(1, 1, sendBuf, receiveBuf, 10, 10, 0);

//发送前加 START, 发送完成后加 STOP，发送 10 个字节
IICSendData(1, 1, sendBuf, 10, 0);

//读取完成后加 STOP，读取 10 个字节
IICRcvData(1, receiveBuf, 10, 0);

if(ICCheckSlaveAddr(0, 0x50, 0) == 1)
{
    //检测到从机地址为 0x50 的从机
}
else
{
    //没有检测到从机地址为 0x50 的从机
}

//寄存器发送，10bit 地址模式，从机地址 0x150，寄存器地址 00 01，发送 10 个字节
IICRegisterSend(1, 0x150, regBuf, sendBuf, 2, 10, 0);

//寄存器读取，10bit 地址模式，从机地址 0x150，寄存器地址 00 01，读取 10 个字节
IICRegisterRead(1, 0x150, regBuf, receiveBuf, 2, 10, 0);

//直接发送，7bit 地址模式，从机地址 0x50，发送 10 个字节
IICDirectSend(0, 0x50, sendBuf, 10, 0);

```

```
//关闭索引值为 0 的转接板 USB  
CloseUsb(0);
```

函数说明：I2C从模式预装数据。通过这个函数将数组预装到转接板，然后I2C主机就能把预装的数据读出。基础版和多电压版每次最多预装1024字节。快速版每次最多预装2048字节。

参数:

*pBuf	预装数据的缓存
len	预装数据的长度：基础版和多电压版每次最多预装1024字节。快速版每次最多预装2048字节。
UsbIndex	USB 索引值（0-99）

返回值:

0	预装数据成功
-1	USB 未打开
小于-1	协议错误

3. I2C 从模式读取数据函数: IICSlaveRcvData

函数原型: int IICSlaveRcvData(unsigned char *rcvBuf, unsigned int len, unsigned int UsbIndex)

函数说明： SPI从模式读取数据。实际转接板已经接收到数据，并存放于转接板的缓存中。这个函数是读取缓存中的数据，最多读取len个字节。如果缓存中数据不够len个字节，则返回实际读取到数据的长度。如果缓存中大于len个字节，则返回len个字节，剩下的数据需要再一次调用IICSlaveRcvData函数才能读出。
基础版和多电压版每次最多读取1024字节。快速版每次最多读取2048字节。

参数:

*rcvBuf	读取数据的缓存
len	预装数据的长度：基础版和多电压版每次最多读取1024字节。快速版每次最多读取2048字节。
UsbIndex	USB 索引值（0-99）

返回值:

大于等于 0	读取数据成功
-1	USB 未打开
小于-1	协议错误

4. 举例

```
//发送的缓存
unsigned char sendBuf[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
//接收的缓存
usngiend char receiveBuf[10];

//打开索引值为 0 的转接板 USB
OpenUsb(0);

//设置 I2C 从模式参数：7bit 地址： 0x50
ConfigIICParamSlave(0, 0x50, 0);

//预装 10 个字节
IICSlavePreloadData(sendBuf, 10, 0);

//接收 10 个字节
IICSlaveRcvData(receiveBuf, 10, 0);

//关闭索引值为 0 的转接板 USB
CloseUsb(0);
```

九 ADC 相关函数

1. ADC 读取函数：GetADCVal

函数原型：int GetADCVal(unsigned char ch,unsigned int UsbIndex)

函数说明：读取ADC通道0和通道1的值，函数执行正确会返回0到4096的12Bit的正整数。
基础版和多电压版的ADC采集范围是0到3.3V时：实际的电压 = 函数返回值*3.3/4096。
当快速版的电压范围是0到3.3V时：实际的电压 = 函数返回值*3.3/4096。
当快速版的电压范围是0到1.8V时：实际的电压 = 函数返回值*1.8/4096。

参数：

ch	ADC通道： 0	使用ADC通道0采集
	1	使用ADC通道1采集

UsbIndex	USB 索引值（0-99）
----------	---------------

返回值:

大于等于 0	ADC 采集成功
-1	USB 未打开
小于-1	协议错误

2. 举例

```

unsigned int ADCVal;

//打开索引值为 0 的转接板 USB
OpenUsb(0);

//使用 ADC 通道 0
ADCVal = GetADCVal(0,0);

//关闭索引值为 0 的转接板 USB
CloseUsb(0);

```

十 IO 口相关函数

1. IO 口读写函数：IOSetAndRead

函数原型: int IOSetAndRead(unsigned int IONum, unsigned int IODir, unsigned int IOBit,unsigned int UsbIndex)

函数说明: 读写IO口函数。这个函数既可以设置IO口输出，也可以读取IO口电平。

参数:

IONum	IO口: 0 1	使用IO0 使用IO1
IODir	IO口方向: 0 1	输入 输出
IOBit	IO口电平: 0	低电平

	1 高电平
UsbIndex	USB 索引值 (0-99)

返回值:

0	IO 输出时，表示设置低电平成功 IO 输入时，表示读取到低电平
1	IO 输出时，表示设置高电平成功 IO 输入时，表示读取到高电平
-1	USB 未打开
小于-1	协议错误

2. 举例

```
//打开索引值为 0 的转接板 USB
OpenUsb(0);

//设置 IO0 输出低电平
IOSetAndRead(0, 1, 0, 0);

//设置 IO0 输出高电平
IOSetAndRead(0, 1, 1, 0);

if (IOSetAndRead(1, 0, 0, 0) == 0)
{
    //读取到 IO1 低电平
}

if (IOSetAndRead(1, 0, 0, 0) == 1)
{
    //读取到 IO1 高电平
}

//关闭索引值为 0 的转接板 USB
CloseUsb(0);
```

十一 PWM 相关函数

1. PWM 输出函数：PWMOut

函数原型：int PWMOut(unsigned int prescaler,unsigned int period,
 unsigned int pulse,unsigned int PulseNum,
 unsigned int UsbIndex)

函数说明：PWM输出函数。这个函数既可以设置PWM输出。PWM输出的最高频率是4. 5M。调用这个函数后会直接输出PWM脉冲。
其中：输出频率 =72M/(预分频*占空比精度)
占空比=(占空比参数/占空比精度)*100%
输出脉冲个数为 0 时，一直输出。
注：输出的脉冲个数是一个近似值，并不是非常精确。

参数：

prescaler	预分频：大于等于36小于等于65535的整数
period	占空比精度：大于等于2小于等于65535的整数
pulse	占空比参数：大于等于1小于等于65535的整数
PulseNum	脉冲数：脉冲数为0时一直输出，最多65535
UsbIndex	USB 索引值（0-99）

返回值：

0	设置 PWM 输出成功
-1	USB 未打开
小于-1	协议错误

2. 关闭 PWM 输出函数：PWMClose

函数原型：int PWMClose(unsigned int UsbIndex)

函数说明：关闭PWM函数。

参数：

UsbIndex	USB 索引值（0-99）
----------	---------------

返回值:

0	关闭 PWM 成功
-1	USB 未打开
小于-1	协议错误

3. 举例

```
//打开索引值为 0 的转接板 USB
OpenUsb(0);

//PWM 输出  输出频率 1KHz， 占空比 50%， 一直输出
PWMOut(72,1000,500,0,0);

//关闭 PWM 输出
PWMClose(0);

//关闭索引值为 0 的转接板 USB
CloseUsb(0);
```

十二 内建列表相关函数（仅快速版支持）

1. 初始化内建列表函数：InternalListInitAllLine

函数原型：int InternalListInitAllLine(unsigned int UsbIndex);

函数说明：初始化内建列表相关内存。

参数:

UsbIndex	USB 索引值（0-99）
----------	---------------

返回值:

0	内建列表初内存始化成功
-1	USB 未打开
小于-1	协议错误

2. 设置内建列表某一行函数：InternalListSetLine

```
函数原型：int InternalListSetLine(unsigned int lineNum,
                                   unsigned int listFun1,
                                   unsigned int listFun2,
                                   unsigned int listParama,
                                   unsigned char* listParamb,
                                   unsigned int listParambLen,
                                   unsigned char* listSend,
                                   unsigned int listSendLen,
                                   unsigned int listDelay1,
                                   unsigned int listRcvLen,
                                   unsigned int listDelay2,unsigned int UsbIndex);
```

函数说明：设置内建列表函数，每次只能设置列表中的一条。列表总共可以设置5条。根据实际需要调用。使用哪条列表，就设置哪条列表，列表可以设置1,2,3,4,5条。
功能1和功能2中红色字体部分为“功能关键字”，功能关键字相同的选项才能在一起使用。如果在功能2中找不到与功能1中相同的关键字，需要填入“null”。
参数a和参数b，在不同的功能中有不同的意义，详见下表。
列表从左向右执行，遇到延时1时延时listDelay1微秒，遇到延时2时延时listDelay2微秒。也就是说延时1是发送和接收之间的延时。延时2是两条列表之间的延时。延时1和延时2的允许的延时时间是0微秒到700000微秒。

参数：

lineNum	列表行号。取值范围 1 到 5	
listFun1	功能 1:	
	LINE_FUN1_SPI_CS0_0_1	执行本行前拉低 CS0，执行完本行后拉高 CS0
	LINE_FUN1_SPI_CS0_1_0	执行本行前拉高 CS0，执行完本行后拉低 CS0
	LINE_FUN1_SPI_CS0_1_1	执行本行前拉高 CS0，执行完本行后拉高 CS0
	LINE_FUN1_SPI_CS0_0_0	执行本行前拉高 CS0，执行完本行后拉高 CS0
	LINE_FUN1_SPI_CS1_0_1	执行本行前拉低 CS1，执行完本行后拉高 CS1
	LINE_FUN1_SPI_CS1_1_0	执行本行前拉高 CS1，执行完本行后拉低 CS1
	LINE_FUN1_SPI_CS1_1_1	执行本行前拉高 CS1，执行完本行后拉高 CS1
	LINE_FUN1_SPI_CS1_0_0	执行本行前拉高 CS1，执行完本行后拉高 CS1
	LINE_FUN1_SPICS_0	单独设置 SPI CS0 脚电平
	LINE_FUN1_SPICS_1	单独设置 SPI CS0 脚电平
	LINE_FUN1_IIC_AddStart	I2C 普通发送和接收，执行本行前加 STRAT
	LINE_FUN1_IIC_NoStart	I2C 普通发送和接收，执行本行前不加 STRAT
	LINE_FUN1_IICd_send	I2C 直接发送
	LINE_FUN1_IICd_read	I2C 直接读取
	LINE_FUN1_IICr_send	I2C 寄存器发送

	LINE_FUN1_IICr_read I2C 寄存器读取 LINE_FUN1_UART_ UART 发送和接收数据 LINE_FUN1_IO_0 IO0 读写 LINE_FUN1_IO_1 IO1 读写 LINE_FUN1_PWM_start PWM 开始 LINE_FUN1_PWM_stop PWM 停止 LINE_FUN1_ADC_0 ADC0 采集 LINE_FUN1_ADC_1 ADC1 采集
listFun2	功能 2: LINE_FUN2_null 无第二功能 LINE_FUN2_SPI_full SPI 全双工 LINE_FUN2_SPI_half SPI 半双工 LINE_FUN2_SPICS_h 拉高 SPI CS 脚 LINE_FUN2_SPICS_l 拉低 SPI CS 脚 LINE_FUN2_IIC_AddStop I2C 普通发送和接收，执行本行后加 STOP LINE_FUN2_IIC_NoStop I2C 普通发送和接收，执行本行后不加 STOP LINE_FUN2_IICd_7bit I2C 直接发送读取 7bit 地址 LINE_FUN2_IICd_10bit I2C 直接发送读取 10bit 地址 LINE_FUN2_IICr_7bit I2C 寄存器发送读取 7bit 地址 LINE_FUN2_IICr_10bit I2C 寄存器发送读取 10bit 地址 LINE_FUN2_IO_w IO 口设置成输出 LINE_FUN2_IO_r IO 口设置成输入
listParama	参数 a: SPI 发送读取数据时: CS 脚延时，十进制，取值范围 0 到 700000，单位微秒，例如: 50 表示延时 50 微秒 设置 SPI CS 脚电平时: 无意义。 I2C 普通发送和接收时: 无意义。 I2C 直接发送读取时: I2C 从机地址，十六进制。 I2C 寄存器发送读取时: I2C 从机地址，十六进制。 UART 发送和接收数据时: 无意义。 IO 读写时: 无意义。 PWM 输出时: 无意义。 ADC 采集时: 无意义。
listParamb	参数 b: SPI发送读取数据时: SPI接收数据时虚拟字节，16进制，1字节长度。 设置 SPI CS 脚电平时: 无意义。 I2C 普通发送和接收时: 无意义。 I2C直接发送读取时: 无意义。 I2C寄存器发送读取时: 寄存器地址，16进制，最多58字节。 UART发送和接收数据时: 发送数据时，在发送数据末尾添加的字节。16进制。例如在发送的数据末尾添加回车换行，则传入0x0D, 0x0A两个字节。 IO 读写时: 无意义。 PWM输出时: 需要传入8个字节。Byte[0]和byte[1]为预分频系数，Byte[2]和byte[3]占空比精度，Byte[4]和byte[5]为占空比参数，Byte[6]和byte[7]输出脉冲数。16进制。 快速版的输出频率=84M/(预分频*占空比精度)。

	占空比=(占空比参数/占空比精度)*100%。 输出脉冲个数为 0 时，一直输出。
listParambLen	参数 b 的长度，字节数。
listSend	发送数据缓存
listSendLen	发送数据的长度：发送数据的长度不能大于 2048，且发送数据长度加接收数据长度之和不能大于 2048。
listDelay1	延时 1：发送和接收之间的延时。单位微秒。最多延时 700000 微秒。
listRcvLen	接收数据的长度：接收数据的长度不能大于 2048，且发送数据长度加接收数据长度之和不能大于 2048。
listDelay2	延时 2：发送和接收之间的延时。单位微秒。最多延时 700000 微秒。
UsbIndex	USB 索引值（0-99）

返回值：

0	内建列表写入成功
-1	USB 未打开
小于-1	协议错误

3. 失能内建列表某一行函数：InternalListDisableLine

函数原型： int InternalListDisableLine(unsigned int lineNum,unsigned int UsbIndex)

函数说明： 失能内建列表某一行。被失能的行将不会执行。

参数：

lineNum	列表行号。取值范围 1 到 5
UsbIndex	USB 索引值（0-99）

返回值：

0	失能内建列表某一行成功
-1	USB 未打开
小于-1	协议错误

4. 测试执行一次内建列表函数：InternalListTsetOnce

函数原型： int InternalListTsetOnce(unsigned int *period,unsigned int UsbIndex)

函数说明： 测试执行一次内建列表，并返回执行一次内建列表所有条目所用的时间，单位微秒。

参数:

*period	返回执行一次内建列表所用的时间,单位微秒
UsbIndex	USB 索引值 (0-99)

返回值:

0	测试执行一次内建列表成功
-1	USB 未打开
小于-1	协议错误

5. I00 跳变沿触发执行内建列表函数: InternalListI00Start

函数原型: `int InternalListI00Start(unsigned int I0Trigger,
unsigned int times,unsigned int UsbIndex)`

函数说明: I00跳变沿触发执行内建列表, 可以设置I00上升沿触发, I00下降沿触发, I00上升沿和下降沿同时触发。可以设置执行次数。设置完成后, 内建列表便会被出发沿触发执行。

参数:

I0Trigger	I00 触发方式: 0 上升沿触发 1 下降沿触发 2 上升沿和下降沿同时触发
times	执行次数: 0 一直执行 大于 0 执行次数
UsbIndex	USB 索引值 (0-99)

返回值:

0	I00 跳变沿触发执行内建列表配置成功
-1	USB 未打开
小于-1	协议错误

6. 定时器沿触发执行内建列表函数: InternalListTimerStart

函数原型: `int InternalListTimerStart(unsigned int period,
unsigned int times,unsigned int UsbIndex)`

函数说明：停止执行内建列表

参数：

UsbIndex	USB 索引值（0-99）
----------	---------------

返回值：

0	停止执行内建列表成功
-1	USB 未打开
小于-1	协议错误

9. 举例

```
//发送的缓存
unsigned char sendBuf[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

//接收的缓存
unsigned char receiveBuf[1000];

//Paramb 参数
unsigned char paramb[58];

//内建列表最小执行时间间隔
unsigned int period;

//打开索引值为 0 的转接板 USB
OpenUsb(0);

//设置 UART 参数：波特率 115200，无校验，1 位停止位
ConfigUARTParam(115200,UART_Parity_No,UART_StopBits_1,0);

//设置 SPI 主模式参数：18M，高位在前，模式 0
ConfigSPIParam(SPI_Rate_18M,SPI_MSB,SPI_SubMode_0,0);

//设置 I2C 主模式参数：时钟频率 400K，时钟延展 10000 个时钟周期
ConfigIICParam(IIC_Rate_400K,10000,0);

//初始化内建列表内存
InternalListInitAllLine(0);

//列表的第一行使用 SPI 口发送和接收数据
//SPI 发送和接收数据，发送数据前拉低 CS0，接收完成后拉高 CS0，SPI 全双工模式，
//CS 延时 50 微秒，接收数据时虚拟字节 0xFF，发送 10 个字节，发送和接收之间不延时，
//接收 10 个字节，接收完成后延时 100 微秒
```

```

paramb[0] = 0xFF;
InternalListSetLine(1, LINE_FUN1_SPI_CS0_0_1, LINE_FUN2_FUN2_SPI_full,
                    50, paramb, 1, sendBuf, 10, 0, 10, 100);

//列表的第二行使用 I2C 口发送数据
//寄存器发送数据, 7bit 地址, 从机地址 0x50, 寄存器地址: 0x00 0x01,
//发送 10 个字节, 发送和接收之间不延时, 接收完成后延时 100 微秒
paramb[0] = 0x00;
paramb[0] = 0x01;
InternalListSetLine(2, LINE_FUN1_IICr_send, LINE_FUN2_IICr_7bit, 0x50, paramb, 2, sendBuf, 10, 0, 0, 100);

//失能内建列表的第 3 行
InternalListDisableLine(3, 0);

//测试执行一次内建列表, 返回最小执行间隔时间到 period 变量
InternalListTsetOnce(&period, 0);

//I00 上升沿触发执行内建列表, 执行 100 次后停止
InternalListI00Start(0, 100, 0);

//定时器触发执行内建列表, 每 1000 微秒执行一次, 一直执行
InternalListTimerStart(1000, 0, 0);

//读取内建列表执行后收到的数据, 读取 1000 个字节
InternalListReceive(receiveBuf, 1000, 0);

//停止执行内建列表
InternalListStop(0);

//关闭索引值为 0 的转接板 USB
CloseUsb(0);

```

北京页贝科技有限公司版权所有